



by Anjali Gopinadhan Nair

Python package security in 2026: How supply chain attacks are targeting your AI development environment

Opinion

Aug 7, 2026 • 5 mins

Your developers trust their tools. That trust is being exploited.



Credit: dTosh / Shutterstock

On March 24, 2026, developers building AI applications with [LiteLLM](https://www.litellm.ai/) — a Python package with 95 million monthly downloads — unknowingly installed malicious code. A threat actor group known as TeamPCP had compromised the PyPI distribution pipeline and pushed malicious versions 1.82.7 and 1.82.8 to the package index. The payload was subtle: a .pth file, a little-known Python mechanism that auto-executes code every time the interpreter starts. If you installed either compromised version, malicious code ran silently — no explicit import needed.

That is not the exception anymore. It is the pattern.

What is actually happening

[ReversingLabs reports](https://www.reversinglabs.com/blog/software-supply-chain-security-report) that malicious open-source packages rose by 73% in 2026. The LiteLLM attack was part of a broader campaign by TeamPCP that systematically compromised widely trusted open-source security tools — including Aqua Security’s Trivy and Checkmarx’s KICS — before moving into AI infrastructure libraries hosted on PyPI.

The attack chain for LiteLLM followed a now-familiar sequence. TeamPCP obtained the maintainer’s PyPI publishing credentials, pushed malicious versions that were virtually indistinguishable from the official package, and embedded a multi-stage payload designed to harvest high-value secrets: AWS, GCP and Azure tokens, SSH keys and cloud account credentials. [According to Zscaler ThreatLabz](https://www.zscaler.com/blogs/security-research/supply-chain-attacks-surge-march-2026), the poisoned packages were available for approximately three hours before quarantine. Three hours was enough to reach tens of thousands of corporate environments.

And it did not stop with LiteLLM. In late April 2026, PyTorch Lightning versions 2.6.2 and 2.6.3 were found to contain credential-stealing malware that executed on import. A single malicious workflow file exposed secrets across entire CI/CD pipelines.

Why AI development environments are uniquely exposed

Most supply chain attacks are bad. Supply chain attacks targeting AI development environments are worse.

CSO Smart Answers [Learn more](#)

Explore related questions

- How does "slopsquatting" exploit non-existent dependencies hallucinated by GPT-4?
- Besides PyPI, which other platforms did TeamPCP target with malicious artifacts?
- Which major entities were impacted by the Europa.eu data breach?
- How is the Pickle serialization format used to poison AI models?
- What happens to the SaaS data stolen by the TeamPCP group?

Ask a question



AI and ML environments blend development, research, cloud infrastructure, data access, model publishing and automation inside the same workspace. A compromised Python package in a standard web application might steal a database credential. The same attack in an AI development environment can expose model weights, training data, cloud tokens across multiple providers, CI/CD pipeline secrets and production API keys — simultaneously, from a single infected dependency.

There is a second layer that most security teams are not accounting for. When developers use AI coding assistants to write code, those assistants frequently suggest pip install directives and import statements that reference specific packages. If the developer trusts the suggestion and installs the named package, and an attacker has already registered a malicious package under that name, the attack succeeds without the attacker ever interacting with the developer directly. Researchers have named this slopsquatting — and **recent research** [<https://arxiv.org/pdf/2605.17062>] found that across nearly 200,000 Python prompts, every major LLM generates hallucinated package names that do not exist on PyPI, creating a persistent attack surface that no individual model update can fully address.

Your developers are not doing anything wrong. They are using the tools that make them productive. The security assumption underneath those tools is broken.

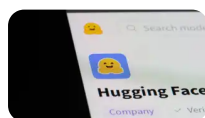
3 controls that matter right now

1. Pin your dependencies and verify integrity

Floating version specifiers — `requests>=2.0` rather than `requests==2.31.0` — allow package managers to silently pull updates that include malicious code. Pin every dependency in

your AI development environments to an exact version and verify checksums against a known-good hash. This alone would have limited the blast radius of the LiteLLM attack to environments that explicitly upgraded to the compromised versions rather than any environment that ran pip install litellm without constraints.

Recommended for you



Malicious Hugging Face model masquerading as OpenAI release hits 244K downloads
By Gyana Swain • 5 mins

[\[https://www.csoononline.com/?post_type=post&p=4169407&utm_source=miso&utm_medium=related&utm_campaign=thumbnail_list\]](https://www.csoononline.com/?post_type=post&p=4169407&utm_source=miso&utm_medium=related&utm_campaign=thumbnail_list)



AI-SPM buyer's guide: 14 tools to secure your AI infrastructure
By david_strom • 19 mins

[\[https://www.csoononline.com/?post_type=post&p=3518733&utm_source=miso&utm_medium=related&utm_campaign=thumbnail_list\]](https://www.csoononline.com/?post_type=post&p=3518733&utm_source=miso&utm_medium=related&utm_campaign=thumbnail_list)



Evidence points to cybercriminals stepping up their AI game
By John Leyden • 6 mins

[\[https://www.csoononline.com/?post_type=post&p=4205861&utm_source=miso&utm_medium=related&utm_campaign=thumbnail_list\]](https://www.csoononline.com/?post_type=post&p=4205861&utm_source=miso&utm_medium=related&utm_campaign=thumbnail_list)

2. Audit post-install hooks in your development pipeline

The LiteLLM attack embedded its payload using Python's .pth file mechanism — code that executes automatically during interpreter initialization, before any import statement runs. Post-install hooks and .pth file manipulation are a documented attack class, but enforcement in developer environments is inconsistent. Require review of packages that include post-install scripts before they reach developer machines. Tools like Socket and Sonatype provide real-time analysis of PyPI packages for malicious behavior before installation. This is not a nice-to-have. Given the pace of AI tooling adoption, it is a basic control.

3. Rotate cloud credentials immediately after any suspected exposure

The LiteLLM payload targeted AWS, GCP and Azure tokens specifically because those credentials provide lateral movement across cloud environments. If your development pipelines pulled LiteLLM during the March 24 exposure window, treat every cloud credential accessible from those environments as potentially compromised and rotate them. Review your cloud provider audit logs for activity patterns that do not correspond to developer-initiated requests — the signature of a stolen token being used by an attacker in a different location.

What this means for security teams

The TeamPCP campaign is not the end of this pattern. It is a proof of concept that AI infrastructure is now a target class. LiteLLM, PyTorch Lightning and the tools in between are packages your AI teams depend on every day. The attackers know that. They know that developers move fast, that AI tooling adoption outpaces security review cycles and that a malicious .pth file is invisible to most endpoint detection products.

The controls above are not complex. They do not require new vendors or new platforms. They require treating Python package installation in AI development environments with the same rigor you apply to production deployments — because in 2026, the distance between a developer’s local environment and your production infrastructure is shorter than it has ever been, and attackers have noticed.

Your developers trust their tools. Make sure that trust is warranted.

Python[https://www.csoonline.com/python/]

Programming Languages[https://www.csoonline.com/programming-languages/]

Software Development[https://www.csoonline.com/software-development/]

Code Security[https://www.csoonline.com/code-security/]

Security[https://www.csoonline.com/security/]

NEWSLETTER

Don't miss a thing

Join the CSO First Look mailing list for the latest news, analysis, and insights. Sign up now!

Email Address

☐ By submitting your information, you agree to our [PRIVACY POLICY \[https://foundryco.com/privacy-policy/\]](https://foundryco.com/privacy-policy/).

Subscribe

About



Policies



More



Our Network



© 2026 FoundryCo, Inc. All Rights Reserved.